

새와 비행기 구별하기: 이미지 학습

7장에서 다루는 내용

- 순방향 신경망 만들기
- Dataset과 DataLoader를 사용한 데이터 로딩
- 분류 손실

6장에서는 경사 하강을 통해 내부 동작을 깊게 이해할 수 있었고, 파이토치가 제공하는 기능을 활용해 모델을 만들고 최적화도 해봤다. 하나의 입력과 하나의 출력을 가진 단순 회귀 모델을 통해 모든 단계를 확실하게 해볼 수 있었지만 재미를 주기에는 다소 모자랐다.

이번 7장에서는 신경망에 대한 기초를 더 다져본다. 이번에는 이미지를 다뤄보겠다. 단언하건대 이미지 인식은 세상 사람들이 딥러닝의 잠재적인 가능성을 깨닫게 된 계기라 할 수 있다.

이번 7장에서는 앞서처럼 단순 신경망을 만들어 간단한 이미지 인식 문제를 단계별로 접근해 나아갈 것이다. 크기는 작지만 상당한 양의 데이터셋을 사용한다. 이미지를 사용할 수 있도록 먼저 데이터셋을 내려받자.

7.1 작은 이미지를 모아 놓은 데이터셋

어떤 주제에 대해 직관적인 이해만큼 좋은 것은 없고 단순한 데이터로 다뤄보는 것만큼 이해하기 쉬운 방법도 없다. 이미지 인식에서 가장 기본이 되는 데이터셋은 MNIST라는 손글씨 숫자 인식 데이터셋이지만 이것만큼 간단하면서도 더 흥미로운 데이터셋을 사용해보겠다. CIFAR-100의 사촌지간쯤 되는 CIFAR-10 데이터셋인데 지난 십여 년간 컴퓨터 비전의 고전이 되어 왔다.

CIFAR-10은 32×32 크기의 컬러(RGB) 이미지 6만 개로 구성되어 있고 1에서 10까지의 정수로 레이블이 매겨져 있다. 0은 비행기, 1은 자동차, 2는 새, 3은 고양이, 4는 사슴, 5는 강아지, 6은 개구리, 7은 말, 8은 배, 9는 트럭이다.¹ 오늘날 CIFAR-10은 새로운 연구를 위한 개발이나 검증으로 사용하기에 너무 단순하지만 학습용으로는 좋다. 데이터셋을 자동으로 내려받고 파이토치 텐서 컬렉션으로 읽어들이기 위해 torchvision 모듈을 사용할 예정이다. CIFAR-10 데이터 일부를 그림 7.1에서 확인할 수 있다.



그림 7.1 CIFAR-10 클래스 이미지 샘플

7.1.1 CIFAR-10 다운로드

CIFAR-10 데이터 다운로드도 마찬가지로 torchvision을 임포트 후 datasets 모듈로 실행한다.

```
# In[2]:  
from torchvision import datasets  
data_path = '../data-unversioned/p1ch7/'  
cifar10 = datasets.CIFAR10(data_path, train=True, download=True)  
cifar10_val = datasets.CIFAR10(data_path, train=False, download=True)  
train = False 설정으로 검증용 데이터를 얻을 수 있으며, 필요시 내려받게 된다.
```

훈련 데이터용 데이터 객체를 만든다.
데이터가 없으면 토치 비전이 알아서
다운로드한다.

CIFAR10 함수의 첫 번째 인자는 데이터를 받을 위치다. 두 번째 인자는 훈련셋인지, 검증셋인지를 지정한다. 세 번째 인자는 파이토치에게 데이터를 찾지 못한 경우 첫 번째 인자로 주어진 위치에 데이터를 내려받으라고 지정한다.

`datasets` 서브모듈은 CIFAR10은 물론이고 MNIST, Fashion-MNIST, CIFAR-100, SVHN, Coco, Omniglot 등의 가장 인기 있는 컴퓨터 비전 데이터셋에 미리 짜여진 절차에 따라 접근할 수 있다. 각 경우에 데이터셋은 `torch.utils.data.Dataset`의 서브클래스로 반환된다. `cifar10` 인스턴스의 메소드를 분석해보면 기본 클래스로 포함된 것을 확인할 수 있다.

```
# In[4]:  
type(cifar10).__mro__
```

```
# Out[4]:  
(torchvision.datasets.cifar.CIFAR10,  
torchvision.datasets.vision.VisionDataset,  
torch.utils.data.dataset.Dataset,  
object)
```


7.1.2 데이터셋 클래스

`torch.utils.data.Dataset`의 서브클래스가 된다는 건 어떤 의미인지 알아보자. 그림 7.2는 파이토치 Dataset이 무엇인지 설명한다. Dataset은 `__len__`과 `__getitem__`을 구현하기 위해 필요한 객체다. `__len__`은 데이터셋의 아이템 수를 반환해야 하고 `__getitem__`은 샘플과 레이블(정수 인덱스)로 이루어진 아이템을 반환해야 한다.²

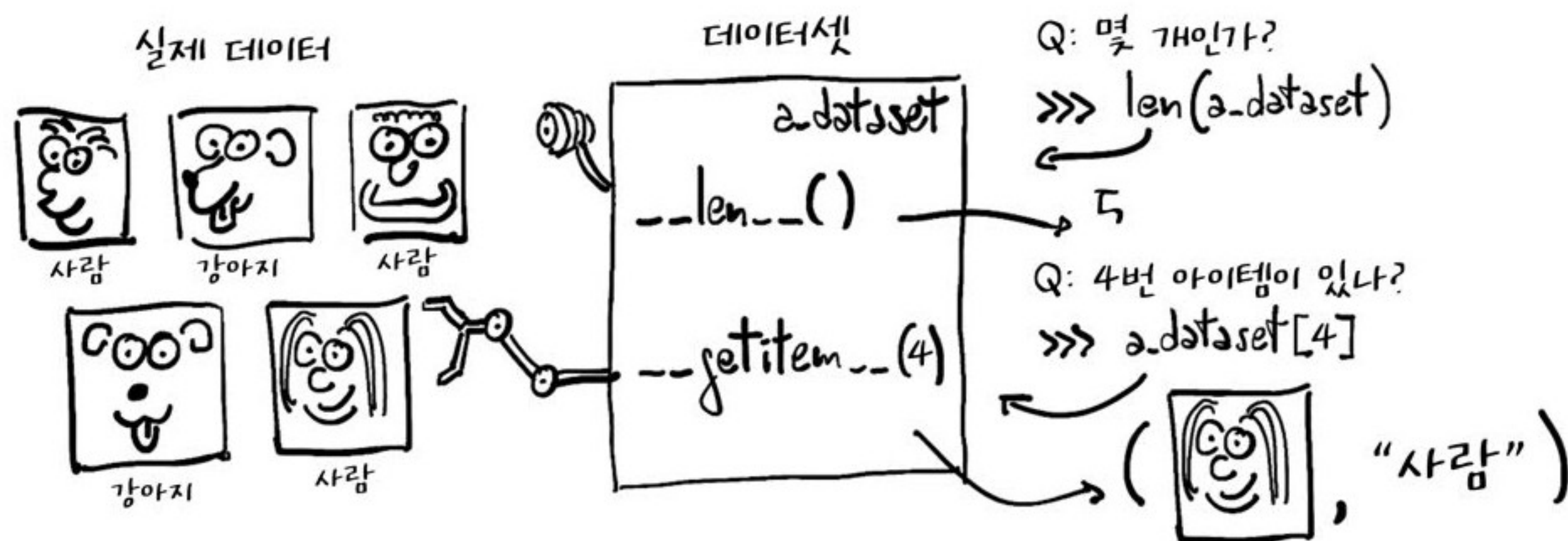


그림 7.2 파이토치 Dataset 객체 개념: 데이터를 직접 들고 있지는 않지만 `__len__`과 `__getitem__`을 통해 일관적으로 접근할 수 있게 해준다.

실제 사용시 `__len__` 메소드가 구현된 파이썬 객체라면 파이썬의 내장 함수인 `len`의 인자로도 사용할 수 있다.

```
# In[5]:  
len(cifar10)
```

```
# Out[5]:  
50000
```

데이터셋에 `__getitem__` 메소드가 구현되어 있으므로 개별 아이템에 접근할 때 표준 서브스크립트^{subscript}에 해당하는 색인용 튜플과 리스트를 사용할 수 있다. 다음은 ‘자동차’에 해당하는 정수값이 1인 PIL(파이썬 이미지 라이브러리^{Python Image Library}) 형식의 이미지를 얻는 코드다.

```
# In[6]:  
img, label = cifar10[99]  
img, label, class_names[label]
```

```
# Out[6]:  
(<PIL.Image.Image image mode=RGB size=32x32 at 0x7FB383657390>,  
1,  
'automobile')
```


data.CIFAR10 데이터셋 샘플은 RGB PIL 이미지 객체라서 바로 그려볼 수 있다.

```
# In[7]:  
plt.imshow(img)  
plt.show()
```

출력 결과를 그림 7.3에서 볼 수 있다. 붉은색 자동차다!³

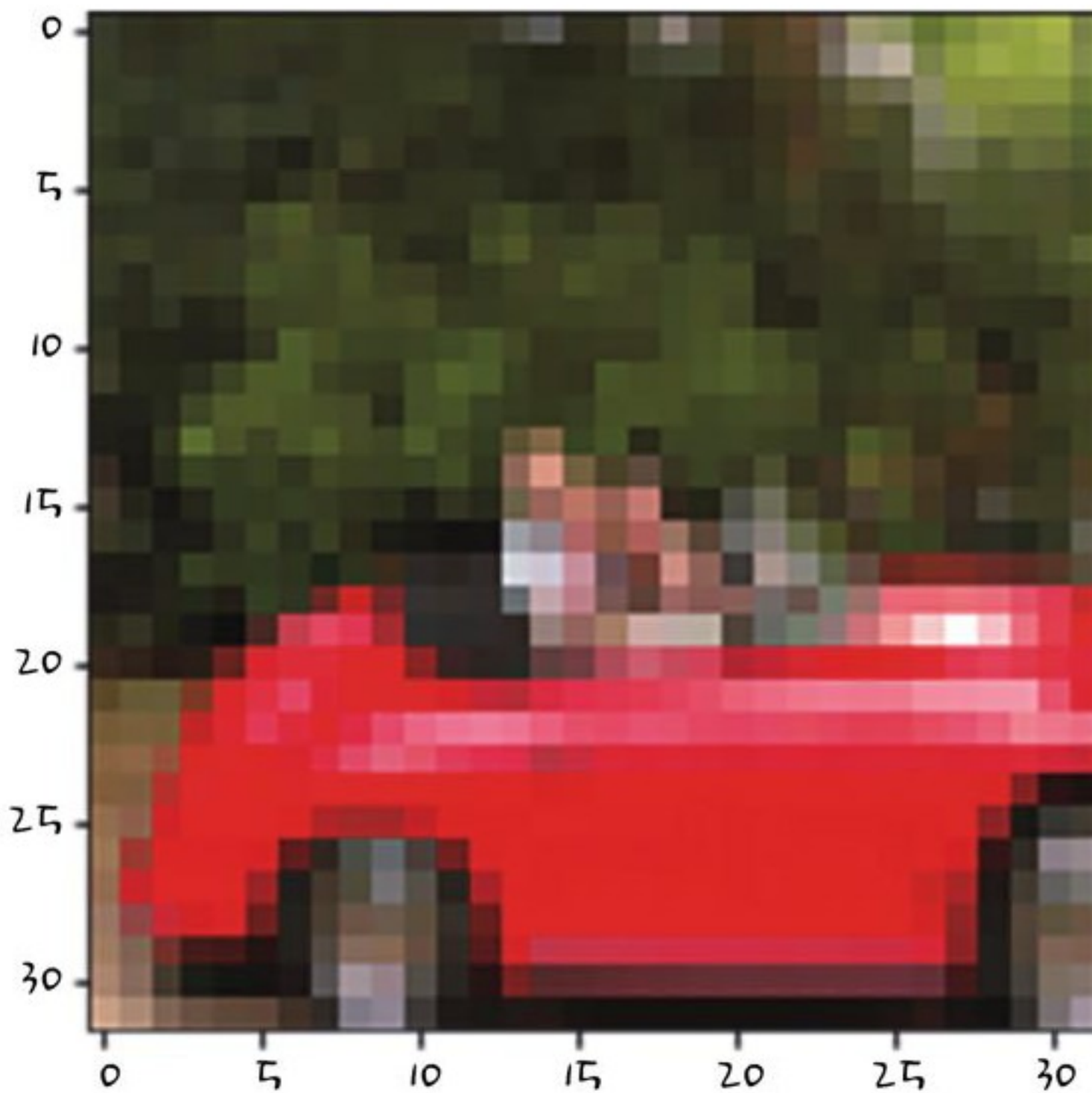


그림 7.3 CIFAR-10 데이터셋의 99번째 이미지: 자동차

7.1.3 데이터 변환

좋다. 이제 PIL 이미지를 파이토치 텐서로 변환할 단계다. `torchvision.transforms`가 필요하다. 이 모듈은 재구성 가능한 함수 형태의 여러 객체를 정의하고 있어서 `datasets.CIFAR10(...)` 같은 `torchvision` 데이터셋에 인자로 전달할 수 있고 로드한 다음에 `__getitem__`가 결과를 반환하기에 앞서 변환을 수행한다. 사용 가능한 객체는 다음과 같다.

```
# In[8]:  
from torchvision import transforms  
dir(transforms)
```

```
# Out[8]:  
['CenterCrop',  
'ColorJitter',  
...  
'Normalize',  
'Pad',  
'RandomAffine',  
...  
'RandomResizedCrop',  
'RandomRotation',  
'RandomSizedCrop',  
...  
'TenCrop',  
'ToPILImage',  
'ToTensor',  
...  
]
```


이 중에 ToTensor라는 게 있는데 넘파이 배열과 PIL 이미지를 텐서로 바꾸는 역할을 한다. 뿐만 아니라 출력 텐서의 차원 레이아웃을 $C \times H \times W$ (4장에서 다룬 채널, 높이, 너비를 뜻한다)로 맞춰 준다.

ToTensor 변환을 사용해보자. 객체로 만들면 PIL 이미지를 인자로 함수처럼 호출할 수 있고 텐서를 리턴한다.

```
# In[9]:
from torchvision import transforms

to_tensor = transforms.ToTensor()
img_t = to_tensor(img)
img_t.shape

# Out[9]:
torch.Size([3, 32, 32])
```

이미지가 $3 \times 32 \times 32$ 텐서로 바뀌었고 이제 RGB 세 개의 채널을 가지는 32×32 이미지가 됐다. label 값은 변동 없이 정수값을 그대로 가진다.

이미 예상했겠지만, 변환 자체를 `dataset.CIFAR10`의 인자로 전달하는 것도 가능하다.

```
# In[10]:
tensor_cifar10 = datasets.CIFAR10(data_path, train=True, download=False,
                                   transform=transforms.ToTensor())
```

이제 데이터셋의 요소를 접근하면 PIL 이미지가 아닌 텐서를 리턴한다.

```
# In[11]:
img_t, _ = tensor_cifar10[99]
type(img_t)
```

```
# Out[11]:
torch.Tensor
```

기대한 것과 같이 채널은 첫 번째 차원에 있고 스칼라 값은 `float32`다.

```
# In[12]:
img_t.shape, img_t.dtype

# Out[12]:
(torch.Size([3, 32, 32]), torch.float32)
```

원본 PIL 이미지 값은 0에서 255(채널당 8비트) 범위인 반면 `ToTensor` 변환으로 데이터가 채널당 32 비트 부동소수점 형태가 되면서 값이 0.0 에서 1.0 사이로 범위가 줄어든다.

```
# In[13]:  
img_t.min(), img_t.max()
```

```
# Out[13]:  
(tensor(0.), tensor(1.))
```

동일한 이미지인지 확인해보자.

```
# In[14]:  
plt.imshow(img_t.permute(1, 2, 0)) ←  $C \times H \times W$ 를  $H \times W \times C$ 로 바꿔준다.  
plt.show()
```

```
# Out[14]:  
<Figure size 432x288 with 1 Axes>
```

그림 7.4에서 볼 수 있듯이 동일한 이미지다.

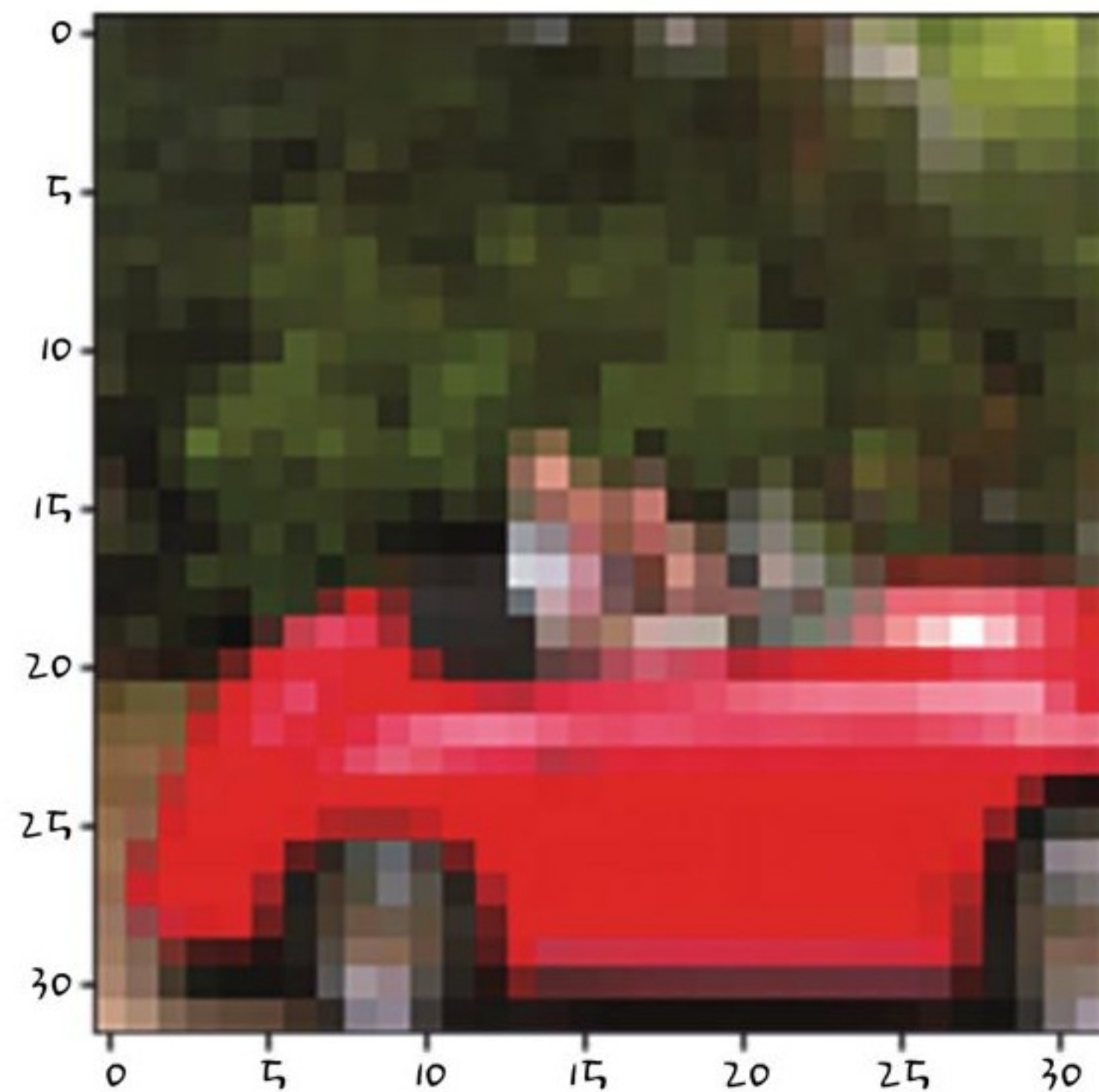


그림 7.4 앞에서 봤던 그림이다.

맷플롯립^{Matplotlib}이 기대하는 $H \times W \times C$ 형태를 위해 $C \times H \times W$ 축을 `permute`를 사용해 변경한 부분에 주의하자.

7.1.4 데이터 정규화

변환 기능은 매우 유용한데 특히 `transforms.Compose`로 여러 변환을 엮어서 사용할 때 더욱 그렇다. 잘 사용하면 정규화와 데이터 증강⁴도 데이터 로딩과 함께 수행할 수 있다. 데이터셋 정규화로 각 채널이 평균값 0과 단위 표준 편차를 가지게 만드는 연습을 하면 도움이 된다. 4장에서 처음 등장했고 5장을 이해했다면 이제 왜 필요한지 알 수 있을 것이다. -1과 1 혹은 -2와 2 사이에서 선형인 활성화 함수를 택하고 데이터를 같은 범위에서 평균을 가지게 한다면 뉴런은 0이 아닌 기울기를 가지게 되므로 빨리 학습할 수 있기 때문이다. 또 각 채널을 정규화해 동일한 분산을 가지게 한다면 채널 정보가 동일한 학습률로 경사 하강을 통해 섞이고 업데이트되는 것도 보장할 수 있다. 5.4.4절에서 다룬 온도 변환 모델에서 가중치를 편향값과 동일한 범위로 만든 이유와 동일하다.

모든 채널이 평균값 0과 단위 표준편차를 가지도록 데이터셋의 각 채널의 평균과 표준편차를 계산해서 $v_n[c] = (v[c] - \text{mean}[c]) / \text{stdev}[c]$ 변환을 적용하면 되는데 이게 바로 `transforms.Normalize`가 하는 일이다. `mean`과 `stdev`는 계산해주지 않으니 따로 계산해야 한다. CIFAR-10 훈련셋으로 계산해보자.

CIFAR-10 데이터셋은 크기가 작기 때문에 필요한 모든 변환을 메모리 내에서 다 할 수 있다. 추가 차원을 만들어 데이터셋이 반환하는 모든 텐서를 쌓아 놓자.

```
# In[15]:
imgs = torch.stack([img_t for img_t, _ in tensor_cifar10], dim=3)
imgs.shape

# Out[15]:
torch.Size([3, 32, 32, 50000])
```

이제 채널별로 평균을 쉽게 계산할 수 있다.

```
# In[16]:
imgs.view(3, -1).mean(dim=1)

# Out[16]:
tensor([0.4915, 0.4823, 0.4468])
```

view(3, -1)은 세 채널은 유지하고 나머지 차원을 적절한 크기 하나로 합친다. 그래서 $3 \times 32 \times 32$ 이미지는 $3 \times 1,024$ 벡터로 바뀌고 평균은 각 채널의 1,024개 요소에 대해 계산하는 것이다.

표준편차 계산도 비슷하다.

```
# In[17]:  
imgs.view(3, -1).std(dim=1)  
  
# Out[17]:  
tensor([0.2470, 0.2435, 0.2616])
```

필요한 값이 있으니 이제 Normalize 변환을 초기화할 수 있다.

```
# In[18]:  
transforms.Normalize((0.4915, 0.4823, 0.4468), (0.2470, 0.2435, 0.2616))  
  
# Out[18]:  
Normalize(mean=(0.4915, 0.4823, 0.4468), std=(0.247, 0.2435, 0.2616))
```

그리고 ToTensor 변환에 이어 붙인다.

```
# In[19]:
transformed_cifar10 = datasets.CIFAR10(
    data_path, train=True, download=False,
    transform=transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize((0.4915, 0.4823, 0.4468),
                               (0.2470, 0.2435, 0.2616))
    ]))
```

이제는 데이터를 그려도 원래와는 많이 다른 모습이다.

```
# In[21]:
img_t, _ = transformed_cifar10[99]

plt.imshow(img_t.permute(1, 2, 0))
plt.show()
```


정규화되어버린 빨간 자동차는 그림 7.5에서 볼 수 있다. 정규화가 RGB 값을 0.0에서 1.0 사이 범위로 만들었고 채널값의 범위도 다 바뀌버렸기 때문이다. 값은 바뀌었지만 데이터는 여전히 존재하며 맷플롯립이 검은색으로 그릴 뿐임을 명심해두자.

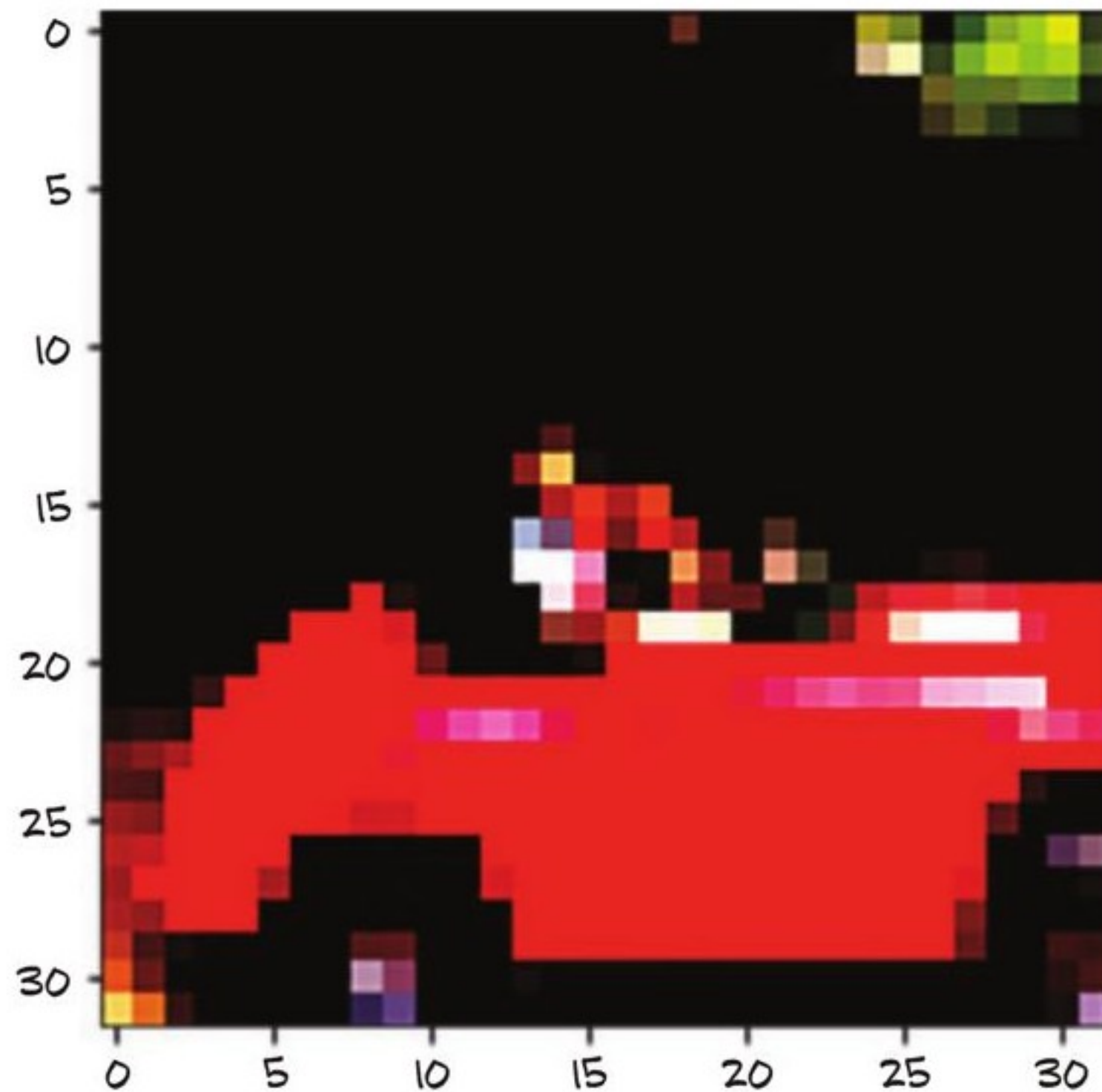


그림 7.5 정규화된 임의의 CIFAR-10 이미지

아직 수만 장의 이미지를 가진 데이터셋을 들고 있다! 필요할 때 편리하게 사용할 수 있다.

7.2 새와 비행기를 구별하기

조류 관찰 클럽 멤버인 제인은 공항 남쪽 숲에 여러 대의 카메라를 설치했다. 카메라는 프레임 안에 뭔가가 들어오면 찍어서 바로 클럽의 실시간 탐조 블로그에 업로드하게 되어 있다. 문제는 새뿐 아니라 날아다니는 비행기도 찍힌다는 점이다.

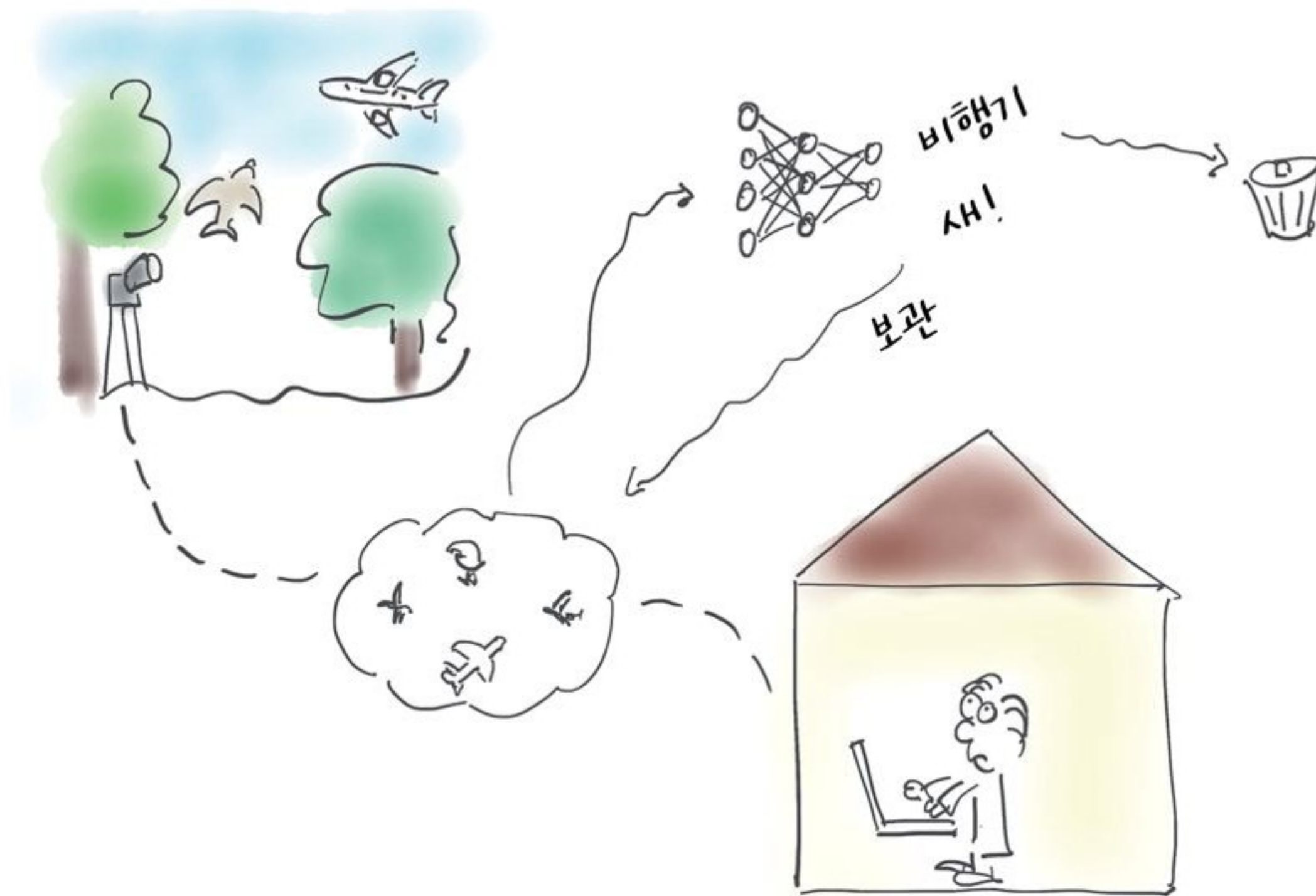


그림 7.6 당면한 문제: 새 관찰 블로그의 원활한 운영을 위해 신경망을 훈련시켜 새와 비행기를 구별할 수 있게 해주자.

제인은 블로그에서 비행기 사진을 삭제하느라 너무 많은 시간을 들이고 있다. 그녀가 필요한 건 그림 7.6과 같은 자동화된 시스템이다. 수동으로 사진을 삭제하는 대신 비행기를 바로 지워 줄 신경망, 즉 마케팅 용어로 'AI'가 필요하다.

걱정할 필요는 없다! 우연히도 여기에 딱 맞는 데이터셋을 가지고 있으니 문제 없다. CIFAR-10 데이터셋에서 새와 비행기를 골라낸 후 신경망을 만들어 새와 비행기를 구분해보자.

7.2.1 데이터셋 구축

가장 먼저 할 일은 데이터의 차원 정보를 맞추는 일이다. 새와 비행기만 들어간 Dataset 서브클래스를 만든다. 하지만 데이터셋은 작으며, 인덱싱과 len만 있으면 되기 때문에 굳이 `torch.utils.data.dataset.Dataset`의 서브클래스일 필요는 없다! `cifar10`에 있는 데이터를 필터링하고 레이블을 다시 매핑해서 연속적으로 만들면 되지 않을까? 다음 코드를 보자.


```
# In[5]:
label_map = {0: 0, 2: 1}
class_names = ['airplane', 'bird']
cifar2 = [(img, label_map[label])
           for img, label in cifar10
           if label in [0, 2]]
cifar2_val = [(img, label_map[label])
               for img, label in cifar10_val
               if label in [0, 2]]
```

cifar2 객체는 Dataset의 기본 요구사항을 만족한다. 즉 `__len__`과 `__getitem__`이 정의되어 있다. 나름대로 머리를 굴려 이렇게 사용하긴 하지만 제한에 걸리는 경우⁵에는 적당하게 Dataset를 만드는 편이 낫다.

이제 데이터셋이 있다! 데이터를 넣을 모델이 필요한 단계다.

7.2.2 완전 연결 모델

앞서 5장에서 신경망을 어떻게 만드는지 배웠다. 간단히 말해 신경망은 피쳐^{feature} 텐서가 들어가고 피쳐 텐서가 나오는 것이다. 생각해보면 이미지도 결국 공간 설정에 따라 적절히 배치된 숫자 집합일 뿐이다. 공간 설정을 어떻게 다뤄야 할지는 모르지만 이미지 픽셀을 받아서 긴 1차원의 벡터로 늘어뜨린다면 결국 일련의 숫자들을 입력 피쳐로 볼 수 있지 않겠는가? 그림 7.7에서 이런 상황을 볼 수 있다.

한번 시도해보자. 샘플마다 피쳐는 얼마나 존재할까? $32 \times 32 \times 3$ 이니까 샘플마다 3,072개의 입력 피쳐가 있는 셈이다. 5장에서 만든 모델에서 출발해 우리의 새로운 모델은 `nn.Linear`이면서 입력 피쳐가 3,072이고 몇 개의 히든 피쳐를 거쳐 활성화 함수로 이어지도록 만들자. 이후 다른 `nn.Linear`로 신경망을 줄여나가면서 원하는 출력 피쳐 수로 맞추자(이 경우에는 2다).

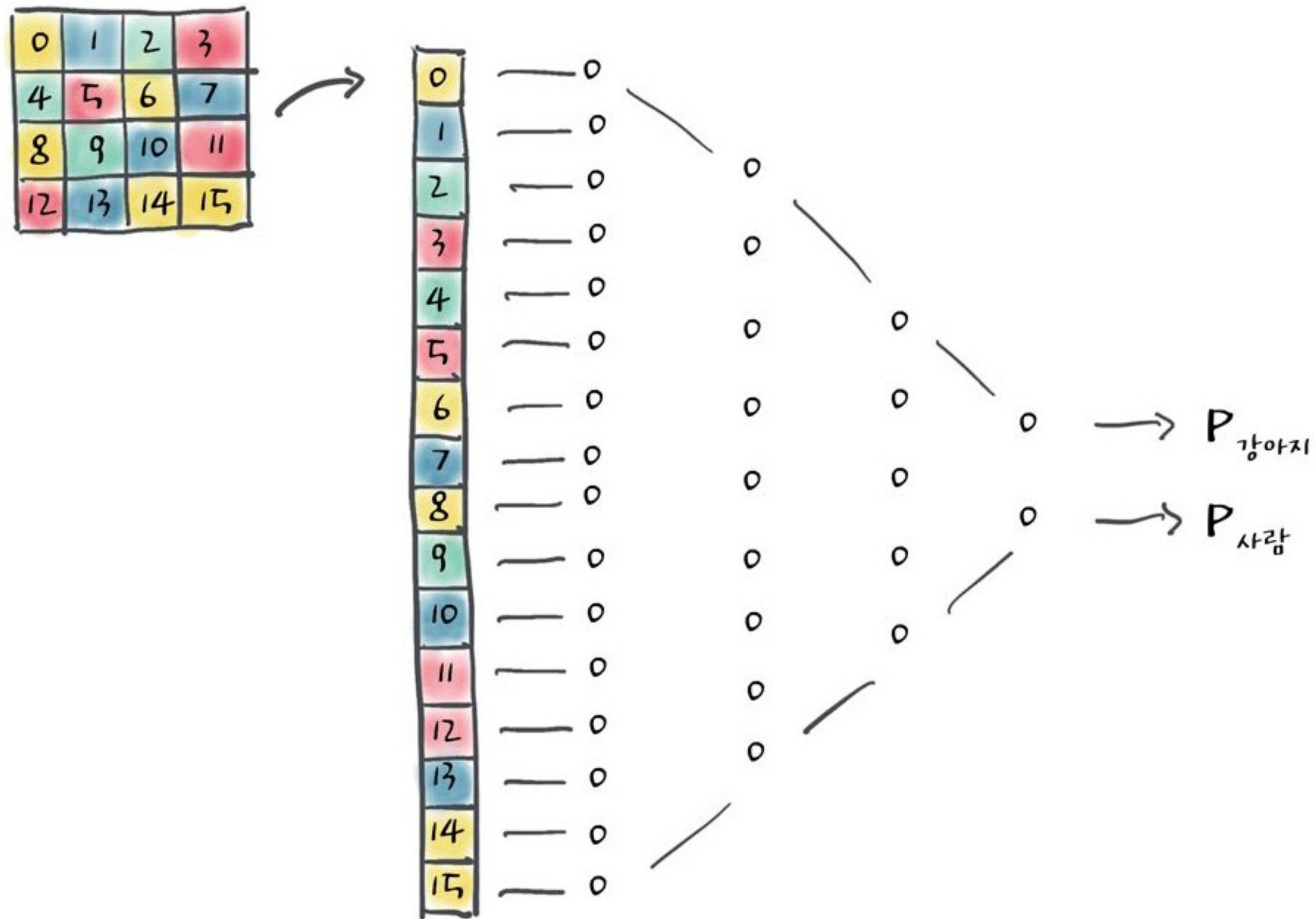


그림 7.7 이미지를 1차원 벡터로 간주하여 이것으로 완전 연결 분류기를 만들자

```
# In[6]:
import torch.nn as nn

n_out = 2

model = nn.Sequential(
    nn.Linear(
        3072,  ← 입력 피쳐
        512,  ← 은닉층 크기
    ),
    nn.Tanh(),
    nn.Linear(
        512,  ← 은닉층 크기
        n_out, ← 출력 클래스
    )
)
```


임의로 512개의 은닉된 피처를 골랐다. 신경망은 최소 하나 이상의 비선형성을 가진 은닉층(활성 함수를 포함하므로 두 개의 모듈)이 필요하다. 6.3절에서 다룬 것처럼 임의의 함수를 학습할 수 있어야 하기 때문이다. 그렇지 않다면 단순 선형 모델에 불과할 뿐이다. 은닉된 피처는 가중치 행렬로 인코딩된 입력값들 간의 (학습된) 관계를 나타낸다. 예를 들어 모델은 학습을 통해 벡터의 176번째 요소와 208번째 요소를 비교할 수도 있다. 하지만 초기에는 벡터 내에서의 구조적인 관계를 모르기 때문에 둘 간의 관계에 집중하지 않는다. 176번째와 208번째 요소는 행렬상에서 5행의 16번째 픽셀과 6행의 16번째 픽셀이기 때문에 인접한 관계임에도 말이다.

이제 모델이 있으니 출력이 어떻게 되어야 할지를 생각해보자.

7.2.3 분류기의 출력

6장에서는 신경망이 (정량적인 의미를 가지는 숫자인) 예측 온도값을 출력으로 만들어냈다. 이번에도 이와 비슷하게 할 수 있다. 신경망이 단일한 스칼라값($n_out = 1$)을 출력하게 만들고 레이블을 부동소수점 수로 변환한 후(비행기는 0.0이고 새는 1.0으로) 이를 `MSELoss`(배치상에서 실측값과의 차이를 제공한 값의 평균)의 타깃으로 사용한다. 이렇게 해서 이 문제를 회귀 문제로 만든다. 그런데 좀더 자세히 생각해보면 사실은 조금 다름을 알 수 있다.⁶

출력값이 카테고리라는 것을 인식할 필요가 있다. 출력값은 새나 혹은 비행기일 뿐이다(원래대로 10개 클래스를 대상으로 했다면 다른 값이 나올 수도 있긴 하다). 4장에서 배운 것처럼 카테고리 변수를 표현할 때는 `[1, 0]`을 비행기로 하고 `[0, 1]`을 새로 하는 것처럼 원핫 인코딩으로 바꿔줘야 한다(순서는 상관없다). CIFAR-10 전체 데이터셋에 대한 클래스 10를 사용하는 경우도 마찬가지다. 이때라면 길이가 10인 벡터를 사용한다.⁷

이상적인 경우 신경망은 비행기에 대해 `torch.tensor([1.0, 0.0])`을 출력하고 새에 대해 `torch.tensor([0.0, 1.0])`을 출력한다. 실제로 우리 분류기는 완벽하지 않으므로 신경망의 출력은 두 값 사이가 될 것이다. 중요한 포인트는 출력을 확률로 해석할 수 있다는 점이다. 첫 번째 값은 ‘비행기’일 확률이고 두 번째 값은 ‘새’일 확률이다.

문제를 확률로 보게 되면 신경망의 출력에 추가적인 제약을 수반한다.

- 출력값의 요소가 가질 수 있는 값은 $[0.0, 1.0]$ 범위로 제한된다(확률은 0보다 작거나 1보다 클 수 없으므로).
- 모든 출력 요소의 값의 합은 1.0이다(결과는 항상 새 아니면 비행기라고 확신한다).

숫자 벡터를 미분 가능하게 해야 하는 것 치고는 가혹한 제한처럼 보일 수 있다. 이런 제약을 극복하면서 미분 가능하게 만드는 똑똑한 방법이 있는데, 바로 **소프트맥스** `softmax`라 부르는 방법이다.

7.2.4 출력을 확률로 표현하기

소프트맥스는 벡터값을 받아 동일한 차원의 다른 벡터를 만드는데, 값이 확률로 표현되어야 하는 제약을 만족한다. 그림 7.8에서 소프트맥스 표현식을 볼 수 있다.

$$0 \leq \frac{e^{x_1}}{e^{x_1} + e^{x_2}} \leq 1$$

각 요소값은 0과 1 사이

$$\frac{e^{x_1}}{e^{x_1} + e^{x_2}} + \frac{e^{x_2}}{e^{x_1} + e^{x_2}} = \frac{e^{x_1} + e^{x_2}}{e^{x_1} + e^{x_2}} = 1$$

모든 요소의 합은 1

$$\text{softmax}(x_1, x_2) = \left(\frac{e^{x_1}}{e^{x_1} + e^{x_2}}, \frac{e^{x_2}}{e^{x_1} + e^{x_2}} \right)$$
$$\text{softmax}(x_1, x_2, x_3) = \left(\frac{e^{x_1}}{e^{x_1} + e^{x_2} + e^{x_3}}, \frac{e^{x_2}}{e^{x_1} + e^{x_2} + e^{x_3}}, \frac{e^{x_3}}{e^{x_1} + e^{x_2} + e^{x_3}} \right)$$
$$\vdots$$
$$\text{softmax}(x_1, \dots, x_n) = \left(\frac{e^{x_1}}{e^{x_1} + \dots + e^{x_n}}, \dots, \frac{e^{x_n}}{e^{x_1} + \dots + e^{x_n}} \right)$$

그림 7.8 소프트맥스 함수

소프트맥스는 벡터의 각 요소 단위로 지수 연산 후 각 요소를 지수 값의 총합으로 나눈다. 코드는 다음과 같다.

```
# In[7]:  
def softmax(x):  
    return torch.exp(x) / torch.exp(x).sum()
```

입력 벡터로 테스트 해보자.

```
# In[8]:  
x = torch.tensor([1.0, 2.0, 3.0])  
  
softmax(x)  
  
# Out[8]:  
tensor([0.0900, 0.2447, 0.6652])
```

다음은 보면 예상한 대로 확률 제한사항을 만족한다.

```
# In[9]:  
softmax(x).sum( )
```

```
# Out[9]:  
tensor(1.)
```

소프트맥스는 단조 함수라서 입력값이 낮아지면 출력값도 따라서 낮아진다. 하지만 범위까지 맞춰주지는 않아서 각 값들 간의 비율이 유지되지 않는다. 입력에서 첫 번째와 두 번째 값의 비율이 0.5라면 출력에서 둘을 비교할 경우 0.3678이 나오는 식이다. 실제로는 모델 파라미터가 학습 과정을 통해 값이 적당한 비율로 맞춰지도록 정해지므로 큰 문제가 되지 않는다.

nn 모듈은 소프트맥스를 모듈처럼 사용할 수 있게 해준다. 통상적으로 입력 텐서는 0번 차원이 배치에 해당하거나 확률 값을 특정 차원에 대해서만 인코딩하므로 nn.Softmax는 소프트맥스 함수를 적용할 차원을 지정하도록 요구한다.

```
# In[10]:
softmax = nn.Softmax(dim=1)

x = torch.tensor([[1.0, 2.0, 3.0],
                  [1.0, 2.0, 3.0]])

softmax(x)

# Out[10]:
tensor([[0.0900, 0.2447, 0.6652],
        [0.0900, 0.2447, 0.6652]])
```


지금 경우는 배치를 다뤘던 때처럼 두 행에 두 개의 입력 벡터가 있다. 그래서 차원 1을 따라 `nn.Softmax`가 동작하도록 초기화했다.

훌륭하다! 이제 모델의 끝에 소프트맥스를 추가하여 신경망이 확률을 출력하도록 만들었다.

```
# In[11]:  
model = nn.Sequential(  
    nn.Linear(3072, 512),  
    nn.Tanh( ),  
    nn.Linear(512, 2),  
    nn.Softmax(dim=1))
```

모델을 훈련시키기 전에도 돌려볼 수는 있으니 한번 해보고 어떤 결과가 나오는지 살펴보자. 새 이미지 하나로 단일 이미지 배치를 만들자(그림 7.9).

```
# In[12]:  
img, _ = cifar2[0]  
  
plt.imshow(img.permute(1, 2, 0))  
plt.show()
```

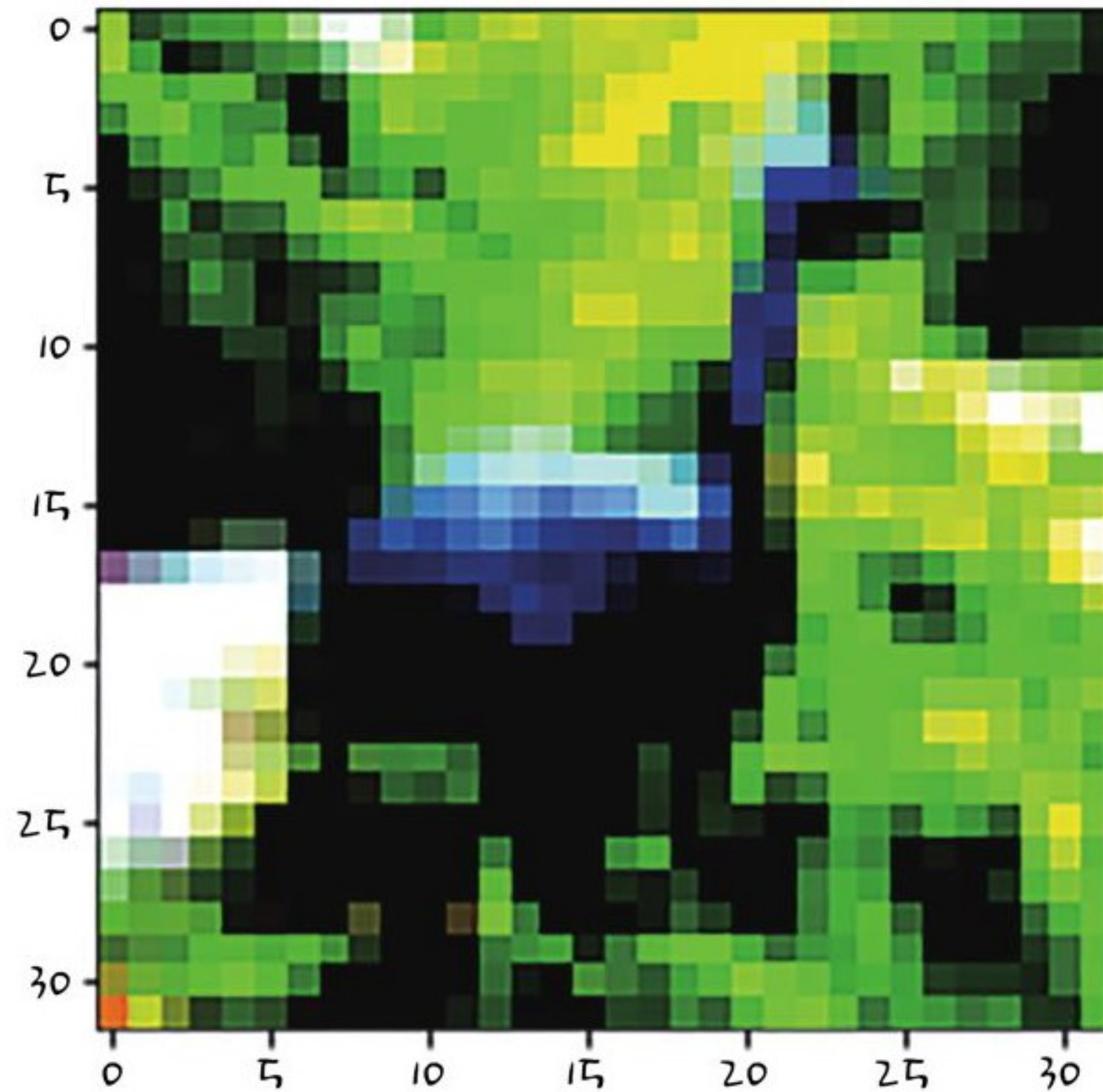


그림 7.9 CIFAR-10 데이터셋에서 임의로 고른 새 이미지(정규화 이후)

아, 잠깐. 모델을 호출하려면 먼저 입력 차원이 맞아야 한다. 우리의 모델은 입력이 3,072개의 피처를 가지고 있으며 0번 차원을 따라 배치로 이뤄지는 데이터를 대상으로 nn에서 작업하게 되어 있다. 따라서 $3 \times 32 \times 32$ 이미지를 1차원 텐서로 만들고 추가 차원을 0번 포지션에 넣어야 한다. 어떻게 하는지는 이미 3장에서 배웠다.

```
# In[13]:  
img_batch = img.view(-1).unsqueeze(0)
```

이제 모델을 호출할 준비가 됐다.

```
# In[14]:  
out = model(img_batch)  
out
```

```
# Out[14]:  
tensor([[0.4784, 0.5216]], grad_fn=<SoftmaxBackward>)
```

드디어 확률을 얻었다! 아직 흥분하기에는 이르다. 우리가 만든 선형 계층의 가중치나 편향값은 전혀 훈련되지 않았고 파이토치에 의해 -1.0과 1.0 사이의 랜덤 값으로 초기화된 상태다. 출력에는 역방향 연산 그래프의 끝에 위치할 출력단을 위한 `grad_fn`도 볼 수 있다.⁸

그리고 지금은 어떤 확률값이 어떤 클래스를 의미하는지 알고 있지만(`class_names`를 생각해 보라) 신경망에서는 이를 표시하지 않는다. 첫 번째는 ‘비행기’, 두 번째는 ‘새’ 혹은 반대였던가? 지금은 신경망에서 구분할 방법이 없다. 두 숫자값에 역전과 후 이런 의미를 부여하는 것은 바로 활성 함수다. 레이블에서 0번 인덱스가 ‘비행기’, 그리고 1번 인덱스는 ‘새’로 주어졌다면 이게 출력값이 의미하는 순서가 된다. 이를 가지고 훈련 후에 출력된 확률에 대해 `argmax` 연산으로 레이블을 얻어낼 수 있다. `argmax`는 제일 높은 확률에 대한 인덱스다. 차원이 주어지는 경우 `torch.max`는 해당 차원에서 가장 높은 요소와 인덱스를 리턴한다. 이제 배치를 가로지르는 대신 확률 벡터 내에서 최댓값을 찾아야 한다. 다음 코드는 차원 1에 대해 수행하고 있다.


```
# In[15]:  
_, index = torch.max(out, dim=1)
```

```
index
```

```
# Out[15]:  
tensor([1])
```

이미지가 ‘새’라고 식별되지만 지금은 우연히 맞춘 것이다. 그래도 이제 확률을 사용하여 모델의 출력을 식별 작업으로 연결해냈다. 이제 모델에 이미지를 넣고 돌려서 연결한 배관 작업에 문제가 없는지 확인할 차례다. 훈련을 할 시간이다. 5장과 6장에서처럼 훈련 중 최소화할 손실 값이 필요하다.

7.2.5 분류를 위한 손실값

손실값은 확률에 의미를 부여한다고 설명한 바 있다. 5장과 6장에서 손실값으로 평균 제곱 오차 (이하 MSE)를 사용했다. 여기에서도 MSE를 사용해서 출력 확률이 $[0.0, 1.0]$ 과 $[1.0, 0.0]$ 에 수렴하도록 만들 수 있다. 하지만 곰곰이 생각해보면, 이제 이 값을 다시 정확하게 만들어내는 것은 크게 중요치 않다. `argmax` 연산으로 예측된 클래스의 인덱스를 뽑아낸 것을 생각해보면, 이제 비행기에 대해서는 첫 번째 확률이 높게 나오고 새에 대해 두 번째 확률이 더 높게 나오는 것이 더 중요한 상황이다. 다시 말해, 정확하게 0.0이나 1.0이 아닌 모든 경우를 애써가며 별점을 주는 대신 분류가 어긋날 경우에 별점을 주고 싶다.

여기서는 `out`이 소프트맥스의 출력이면서 `class_index`가 ‘비행기’일 때는 0이고 ‘새’일 때는 1을 포함하는 벡터라고 할 경우, 이제 정답 클래스 `out[class_index]`와 관련된 확률을 극대화할 필요가 있다. 정답 클래스에 대한 확률 수치를 (데이터가 주어졌을 때 모델의 파라미터에 대한) **가능도** `likelihood`라고 한다.⁹ 바꿔 말하면 가능도가 낮을 때, 즉 다른 클래스의 확률이 매우 높을 때 값이 커지는 손실 함수가 필요하다. 반대로 가능도가 다른 클래스보다 높으면 손실값은 낮아야 한다. 확률값을 억지로 1까지 끌어올리는 데 집착할 필요가 없다.

이런 식으로 동작하는 손실 함수가 있다. 바로 NLL^{Negative Log Likelihood} 함수다. 표현식은 $NLL = -\sum(\log(\text{out}_i[c_i]))$ 이며 $\sum$ 은 N 개의 샘플 합이고 c_i 는 샘플 i 에 대한 정답 클래스이다. 그림 7.10은 예측 확률의 함수로서 NLL을 그린 것이다.

이 그림을 살펴보면 데이터에 낮은 확률이 주어질 경우 NLL은 무한으로 늘어나는 반면, 확률이 0.5보다 커지는 경우 NLL은 완만하게 감소하는 경향을 볼 수 있다. NLL은 확률을 입력으로 받기 때문에 가능도가 증가하면 다른 확률은 필연적으로 줄어든다는 점을 기억하자.

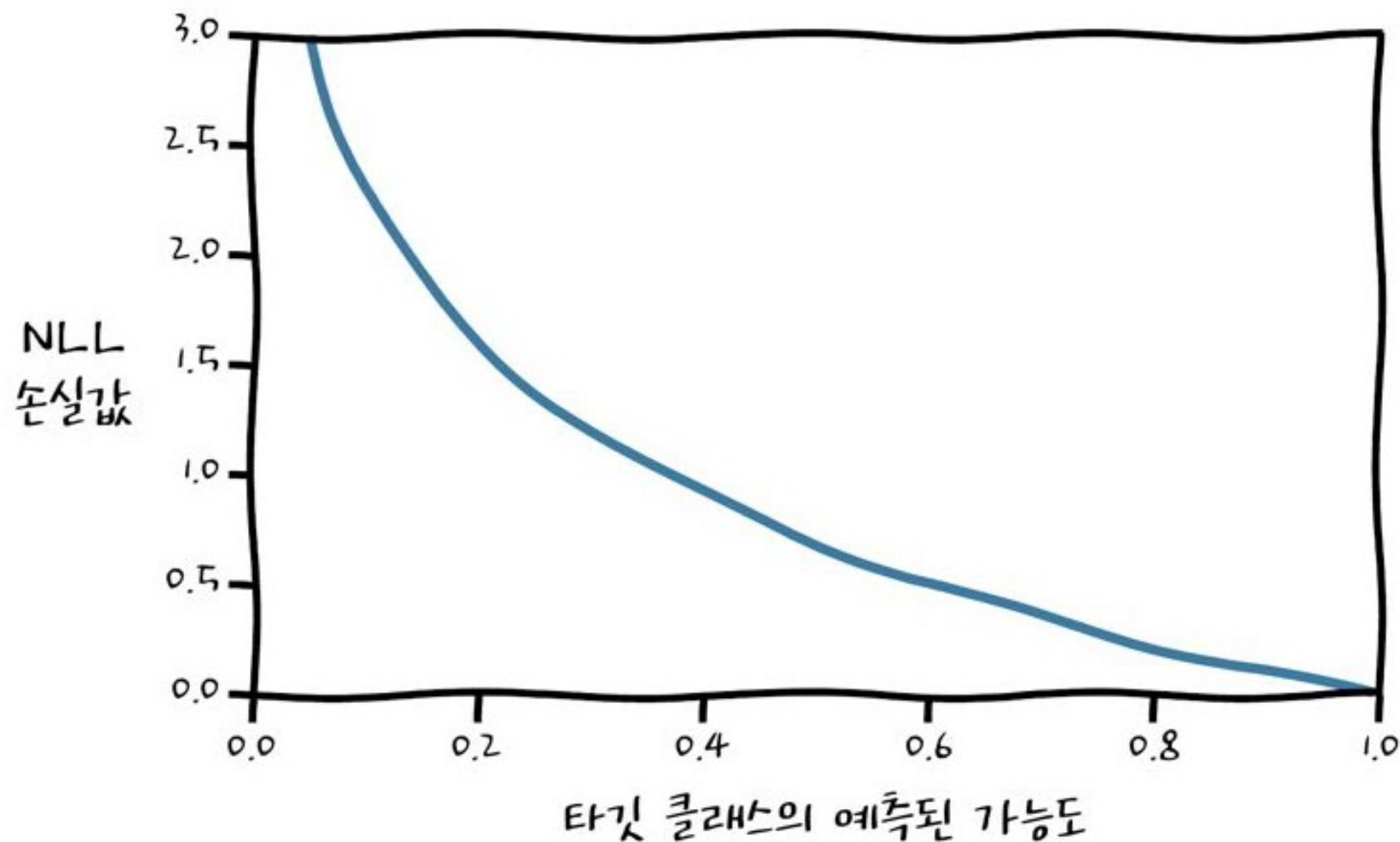


그림 7.10 예측 확률 함수로서의 NLL 손실값

요약하면 분류를 위한 손실은 다음과 같이 계산할 수 있다. 배치 내의 각 샘플에 대해,

1. 순방향 전달 후 마지막 (선형) 계층에서 출력값을 얻는다.
2. 이들의 소프트맥스 값을 계산하여 확률을 얻는다.
3. (파라미터의 가능도에 해당하는) 정답 클래스와 일치하는 예측 확률값을 얻는다. 지도 supervised 문제이기 때문에 실측값이 존재하므로, 정답 클래스를 안다고 가정할 수 있다.
4. 로그값을 계산해서 앞에 마이너스를 붙인 다음 손실값에 더한다.

파이토치로는 이 과정을 어떻게 처리할까? 파이토치는 `nn.NLLLoss` 클래스를 제공한다. 하지만 기대와 달리, 확률 대신 로그 확률의 텐서를 받는다. 그리고 데이터 배치에 대한 모델의 NLL을 계산한다. 입력 형태가 이런 형식인 데에는 나름의 이유가 있다. 입력을 확률의 로그값으로 받으면, 확률이 0에 가까울 경우 문제가 되는데, 이를 해결하기 위해 `nn.Softmax` 대신 `nn.LogSoftmax`를 사용함으로써 계산이 안정적으로 수행되게 하는 것이다.

모델을 수정하여 `nn.LogSoftmax`를 출력 모듈로 사용하자.

```
model = nn.Sequential(  
    nn.Linear(3072, 512),  
    nn.Tanh(),  
    nn.Linear(512, 2),  
    nn.LogSoftmax(dim=1))
```

그리고 NLL 손실값을 초기화한다.

```
loss = nn.NLLLoss()
```

손실값은 배치에 대한 nn.LogSoftmax 출력을 첫 번째 인자로 받고 클래스 인덱스(여기서는 여러 개의 0이나 1) 텐서를 두 번째 인자로 받는다. 이제 새를 가지고 테스트해보자.

```
img, label = cifar2[0]
```

```
out = model(img.view(-1).unsqueeze(0))
```

```
loss(out, torch.tensor([label]))
```

```
tensor(0.6509, grad_fn=<NllLossBackward>)
```


이렇게 손실값을 알아보고 나니, 크로스엔트로피 손실을 사용하는 것이 MSE보다 얼마나 개선되는지를 파악할 수 있다. 그림 7.11을 살펴보면 크로스엔트로피는 예측이 타깃(손실이 적은 모서리에서 정답 클래스는 99.97%의 예측 확률을 보인다)에서 멀어지는 경우 그래프의 경사가 적당히 생기지만 이번에 사용하지 않은 MSE는 훨씬 더 일찍 포화되어 좋지 않은 예측을 하게 된다. 근본적으로 MSE의 기울기는, 잘못된 예측에 대한 소프트맥스 함수의 평탄화를 보상하기에 너무 작다는 사실을 알 수 있다. 이렇게 때문에 분류 문제에서 확률에 대해 MSE를 사용하는 건 좋은 방법이 아니다.

성공적인 분류 손실값과 그렇지 못한 경우

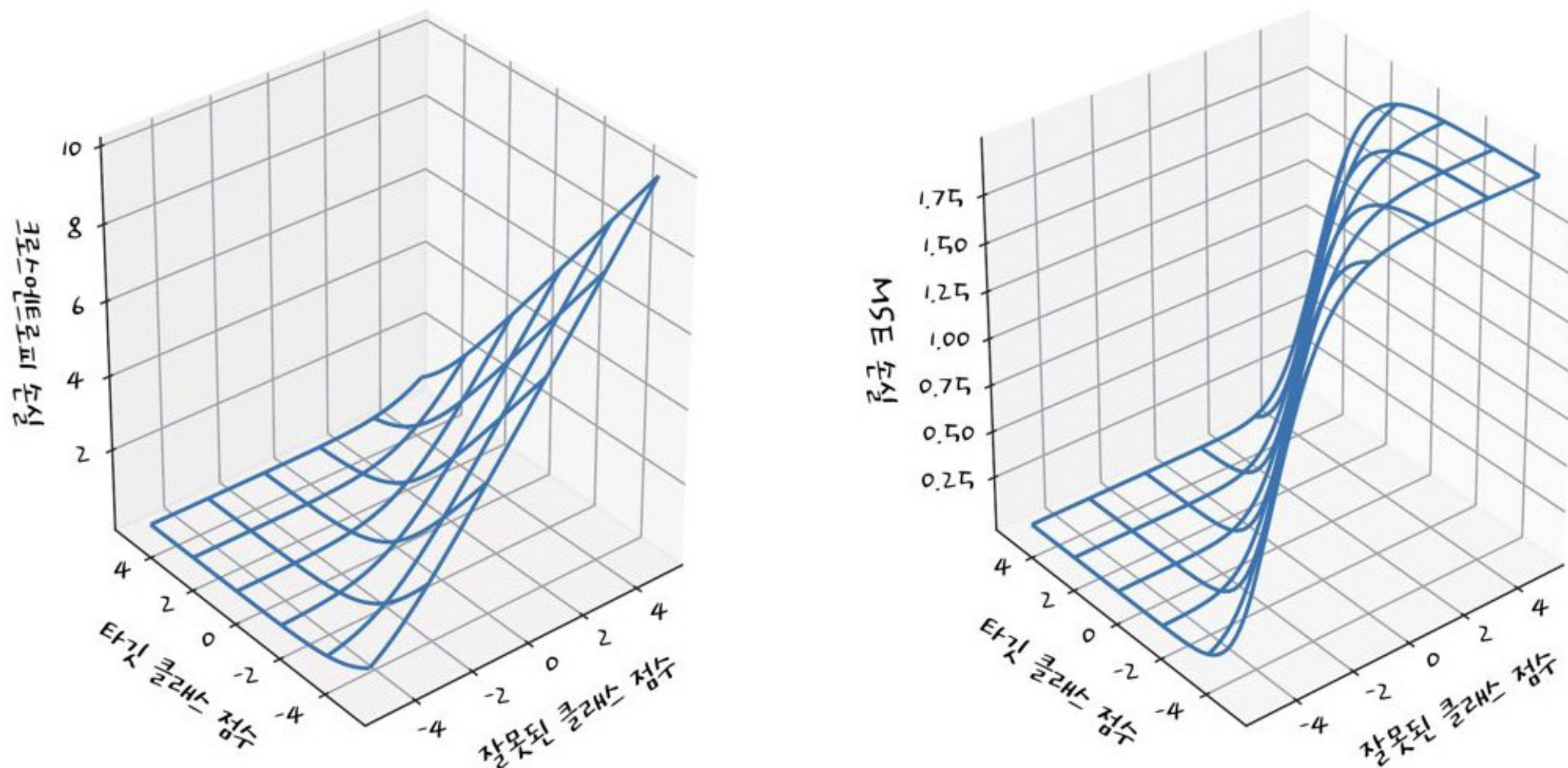


그림 7.11 크로스엔트로피(왼쪽)와 예측 확률과 타깃 확률 벡터 사이의 MSE(오른쪽)를 (로그)소프트맥스를 거치기 전 예측 점수의 함수로 나타낸 그림

7.2.6 분류기 훈련

좋다! 5장에서 작성한 훈련 루프를 가져와서 어떻게 훈련시킬지 알아보자(과정은 그림 7.12에서 볼 수 있다).

```
import torch
import torch.nn as nn

model = nn.Sequential(
    nn.Linear(3072, 512),
    nn.Tanh(),
    nn.Linear(512, 2),
    nn.LogSoftmax(dim=1))

learning_rate = 1e-2

optimizer = optim.SGD(model.parameters(), lr=learning_rate)
loss_fn = nn.NLLLoss()

n_epochs = 100

for epoch in range(n_epochs):
    for img, label in cifar2:
        out = model(img.view(-1).unsqueeze(0))
        loss = loss_fn(out, torch.tensor([label]))

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    print("Epoch: %d, Loss: %f" % (epoch, float(loss)))
```

마지막 이미지의 손실값을 출력한다.
다음 8장에서는 전체 에포크의 평균을
출력하도록 개선할 예정이다.

자세히 보면 훈련 루프에 살짝 변경을 가했다. 5장에서는 에포크 전체에 하나의 루프가 있었다 (훈련셋의 모든 샘플을 평가해야 에포크가 끝났음을 상기하자). 지금은 하나의 단일 배치에 1만 개의 이미지를 모두 평가하는 것은 너무 많으므로 내부 루프 안에서 한 번에 하나의 샘플을 평가하고 단일 샘플에 대해 역전파해보자.

Ⓐ

for n epochs:

with every sample in dataset:

evaluate model (forward)

compute loss

accumulate gradient of loss

(backward)

update model with accumulated gradient

Ⓑ

for n epochs:

with every sample in dataset:

evaluate model (forward)

compute loss

compute gradient of loss

(backward)

update model with gradient

Ⓒ

for n epochs:

split dataset in minibatches

for every minibatch:

with every sample in minibatch:

evaluate model (forward)

compute loss

accumulate gradient of loss (backward)

update model with accumulated gradient

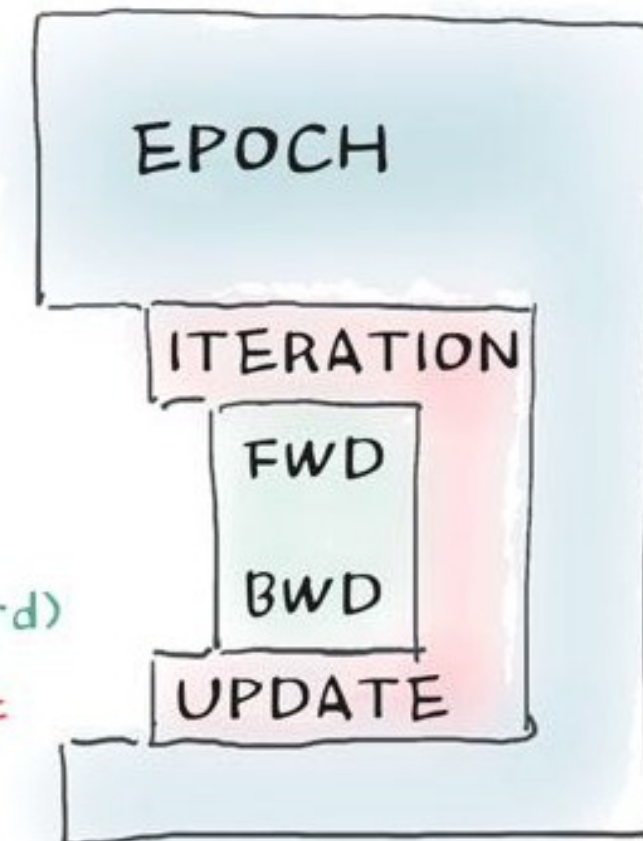


그림 7.12 훈련 루프를 표현한 수도(pseudo) 코드. (A) 모든 데이터셋에 대한 평균으로 조정. (B) 각 샘플마다 모델을 조정. (C) 미니 배치마다 평균을 구해서 조정

첫 번째 경우에서 기울기값은 적용하기 전에 모든 샘플로부터 값이 누적되지만 지금은 단일 샘플에서 부분적으로 기울기를 얻어 파라미터를 변경한다. 그런데 하나의 샘플에 기반해서 손실 값을 줄이는 게 다른 샘플에 대해서는 안 좋아진 것일 수 있다. 그래서 각 에포크마다 샘플을 섞은 후 한 번에 하나 혹은 (안정성을 고려해서) 여러 개의 샘플에 대해 기울기를 평가하면 경사 하강에 랜덤한 효과를 넣어줄 수 있다. SGD를 기억하고 있는가? SGD^{stochastic gradient descent}의 S는 확률적이라는 의미로서, 섞은 데이터의 작은 배치(미니 배치) 단위로 수행한다. 미니 배치에서 얻은 기울기를 사용하면 전체 데이터셋에 대한 값만큼 근사^{approximation}하지는 않지만 훈련 중에 만날지도 모르는 지역 최솟값^{local minima}에 최적화 프로세스가 빠져버리는 것을 예방하고 수렴을 돕는 것으로 알려져 있다. 그림 7.13에서 설명하는 것처럼 미니 배치에서의 기울기는 이상적인 궤적에서 랜덤하게 떨어져 있다. 그래서 이 경우 우리는 작은 학습률을 사용한다. 각 에포크에서 데이터셋을 섞으면 미니 배치에서 얻은 기울기의 시퀀스가 전체 데이터셋에서 계산한 기울기를 대표하도록 만드는 데 도움이 된다.

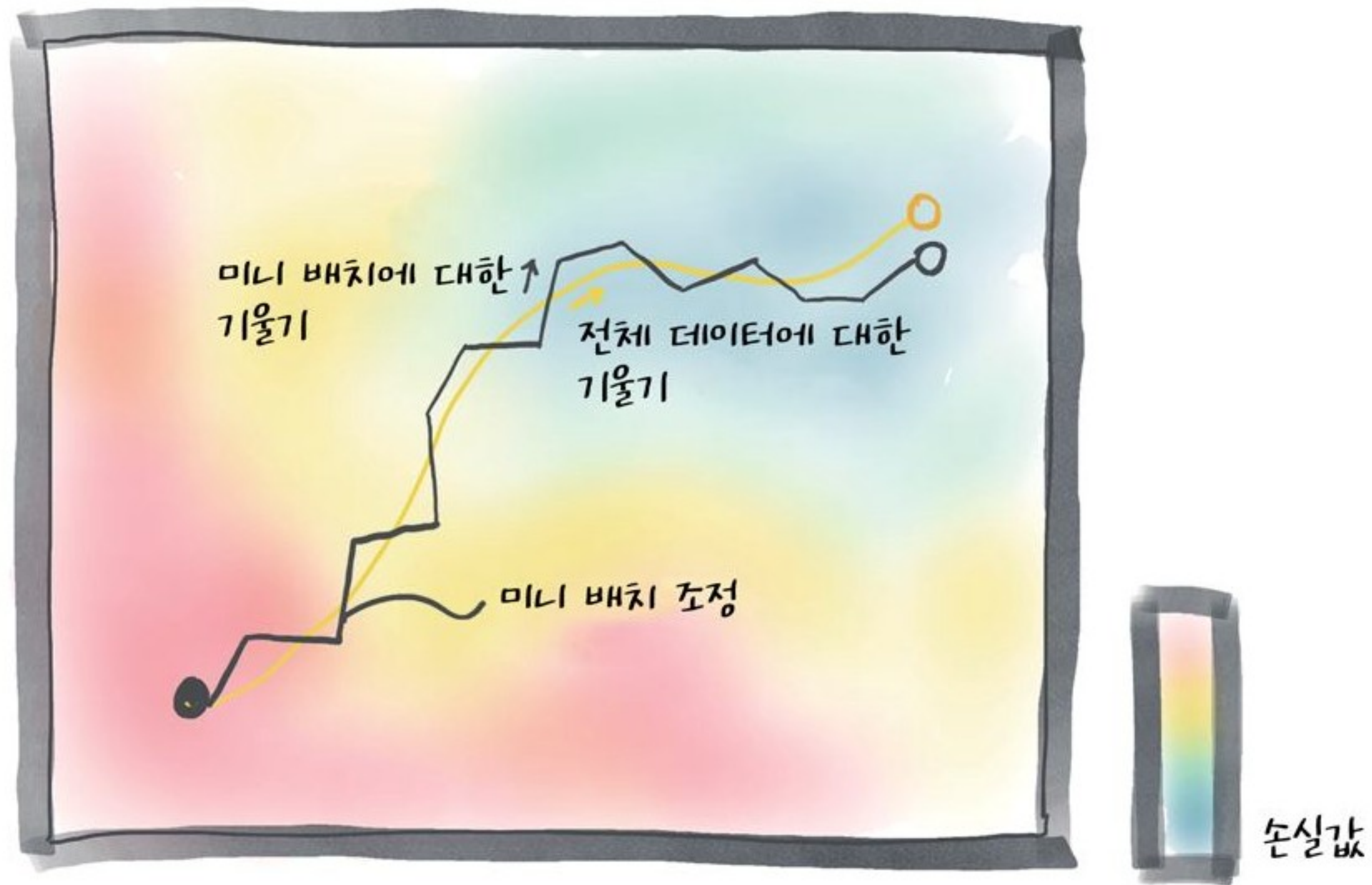


그림 7.13 전체 데이터셋에 대해 평균을 구한 경사 하강(엷은 색)과 SGD를 사용해 랜덤하게 뽑은 미니 배치에서 예측된 경사 하강(짙은 색)의 비교

통상 미니 배치 크기는 학습률을 지정했던 것과 같이 훈련 전에 상수로 고정한다. 이와 같은 고정 값들은 모델의 파라미터와 구분하여 하이퍼파라미터(hyperparameters)라 부른다.

우리의 훈련 코드는 데이터셋에서 한 번에 하나의 아이템을 골라 크기가 1인 미니 배치를 만든다. torch.utils.data 모듈에는 미니 배치의 데이터를 섞거나 구조화하는 작업을 돕는 DataLoader 클래스가 있다. 데이터 로더를 활용해 데이터셋에서 미니 배치에 포함될 샘플을 가져올 때 여러 가지 샘플링 전략 중에서 하나를 고를 수 있다. 가장 흔한 전략은 각 에포크마다 데이터를 섞은 후 고르게 샘플링하는 것이다. 그림 7.14는 데이터 로더가 Dataset으로부터 데이터를 얻은 후 인덱스를 섞는 모습을 보여준다.

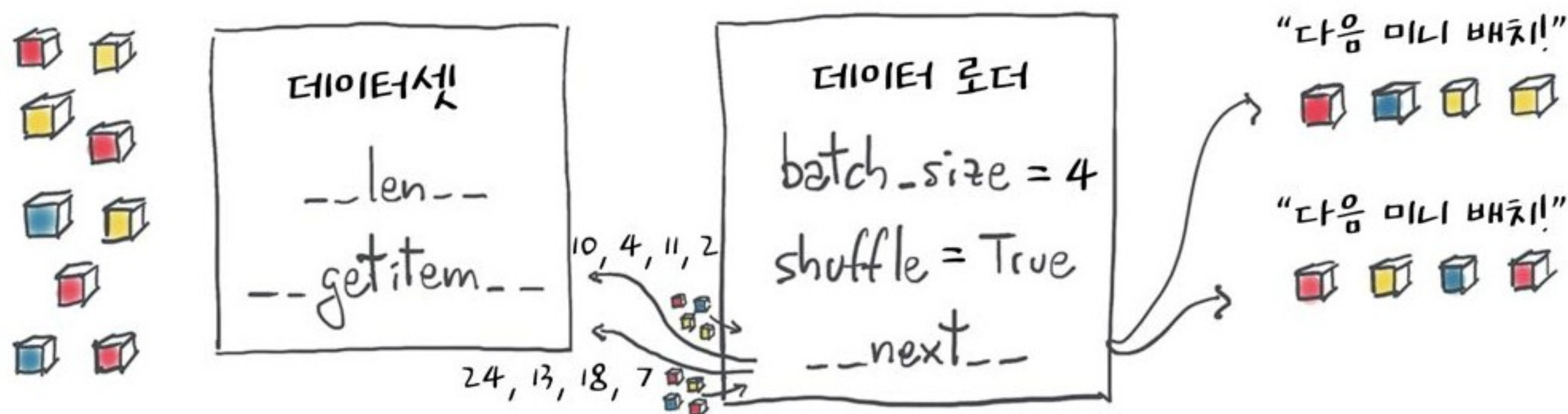


그림 7.14 데이터 로더가 데이터셋에서 개별 데이터 아이템을 샘플링하여 미니 배치를 구성하는 모습

이를 어떻게 구현하는지 살펴보자. 먼저 DataLoader 생성자는 입력으로 Dataset 객체와 함께 batch_size와 shuffle을 입력받는다. shuffle은 불리언 값으로 데이터가 각 에포크마다 섞여야 할지를 지정한다.

```
train_loader = torch.utils.data.DataLoader(cifar2, batch_size=64, shuffle=True)
```

DataLoader는 순회 가능하기 때문에 다음과 같은 새로운 훈련 코드의 루프 안에 바로 넣어 사용할 수 있다.

```

import torch
import torch.nn as nn

train_loader = torch.utils.data.DataLoader(cifar2, batch_size=64, shuffle=True)

model = nn.Sequential(
    nn.Linear(3072, 512),
    nn.Tanh(),
    nn.Linear(512, 2),
    nn.LogSoftmax(dim=1))

learning_rate = 1e-2

optimizer = optim.SGD(model.parameters(), lr=learning_rate)

loss_fn = nn.NLLLoss()

n_epochs = 100

for epoch in range(n_epochs):
    for imgs, labels in train_loader:
        batch_size = imgs.shape[0]
        outputs = model(imgs.view(batch_size, -1))
        loss = loss_fn(outputs, labels)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    print("Epoch: %d, Loss: %f" % (epoch, float(loss)))

```

데이터를 섞었으므로 이제 랜덤 배치에 대한 손실값을 출력한다.
 이 코드부는 8장에서 더 개선할 예정이다.

루프 내부에서 `imgs`는 $64(32 \times 32)$ RGB 이미지에 해당하는, 크기 $64 \times 3 \times 32 \times 32$ 인 텐서이며, `labels`는 레이블 인덱스를 가진 크기 64의 텐서다.

훈련 코드를 실행해보자.

```
Epoch: 0, Loss: 0.523478
Epoch: 1, Loss: 0.391083
Epoch: 2, Loss: 0.407412
Epoch: 3, Loss: 0.364203
...
Epoch: 96, Loss: 0.019537
Epoch: 97, Loss: 0.008973
Epoch: 98, Loss: 0.002607
Epoch: 99, Loss: 0.026200
```

어찌 됐든 손실값이 줄어들고 있다. 다만 이 값을 충분히 줄일 방법은 아직 모른다. 일단 지금 작업의 목적은 이미지를 정확한 클래스로 분류하는 일이고 되도록이면 독립된 데이터셋에서 진행되길 희망하므로 검증셋으로 테스트하여 전체 중에 정확하게 분류된 숫자를 세어 모델의 정확도를 계산할 수 있다.

```
val_loader = torch.utils.data.DataLoader(cifar2_val, batch_size=64, shuffle=False)
```

```
correct = 0
```

```
total = 0
```

```
with torch.no_grad():
```

```
    for imgs, labels in val_loader:
```

```
        batch_size = imgs.shape[0]
```

```
        outputs = model(imgs.view(batch_size, -1))
```

```
        _, predicted = torch.max(outputs, dim=1)
```

```
        total += labels.shape[0]
```

```
        correct += int((predicted == labels).sum())
```

```
print("Accuracy: %f", correct / total)
```

```
Accuracy: 0.794000
```

훌륭하다고 할 순 없지만 맘대로 예측하는 것보다는 낫다. 정말 단순한 분류기임에도 불구하고, 동작한다는 것 자체가 기적이다. 오히려 사용한 데이터셋이 너무 단순해서 이럴 수 있다. 예를 들어 각 클래스가 배경색처럼 구조적으로 다른 부분이 있어서 몇 픽셀에 기반하여 새와 비행기를 구별하고 있을지도 모른다.

모델에 계층을 더 추가하면 모델의 깊이나 용량이 늘어나게 된다. 예를 들어 다음처럼 만들 수 있다.

```
model = nn.Sequential(  
    nn.Linear(3072, 1024),  
    nn.Tanh( ),  
    nn.Linear(1024, 512),  
    nn.Tanh( ),  
    nn.Linear(512, 128),  
    nn.Tanh( ),  
    nn.Linear(128, 2),  
    nn.LogSoftmax(dim=1))
```

중간 계층의 줄어드는 중간 결과에 정보가 잘 압축되기를 바라며 출력단으로 갈수록 피쳐 수를 점점 줄여봤다.

`nn.LogSoftmax`와 `nn.NLLLoss` 조합은 `nn.CrossEntropyLoss`와 동일하다. `nn.CrossEntropyLoss`는 파이토치의 특징을 잘 나타내는 요소다. `nn.NLLLoss`는 크로스엔트로피를 계산하지만 입력으로 로그 확률 예측을 받는 반면, `nn.CrossEntropyLoss`는 (logits라 부르는) 점수를 입력으로 받는다. 구체적으로 말하면 `nn.NLLLoss`는 타겟에 모든 것을 담아 넣는 디랙 분포^{Dirac distribution}와 로그 확률 입력으로 주어지는 예측 분포 사이의 크로스엔트로피라고 볼 수 있다.

더 복잡한 얘기이긴 한데, 정보 이론 중 샘플 크기로 정규화하는 부분에서 이 크로스 엔트로피는 타겟의 분포에 대한 예측 분포로서 ‘음의 로그 가능도^{negative log likelihood}’로 해석할 수 있다. 그래서 두 손실값 모두 우리의 모델이 (소프트맥스가 적용된) 확률을 예측한다고 할 때 주어진 데이터에 대한 모델 파라미터의 음의 로그 가능도라 할 수 있다. 이 책에서 이런 세부 사항을 꼭 알아야 하지는 않지만 최소한 문헌에서 볼 수 있는 이런 용어들을 파이토치에서 사용할 때 여러 분이 혼란스러워 하지는 않아야 한다.

일반적으로 신경망의 마지막 계층에서는 nn.LogSoftmax 대신 손실값으로 nn.CrossEntropyLoss를 사용한다.

```
model = nn.Sequential(  
    nn.Linear(3072, 1024),  
    nn.Tanh(),  
    nn.Linear(1024, 512),  
    nn.Tanh(),  
    nn.Linear(512, 128),  
    nn.Tanh(),  
    nn.Linear(128, 2))  
  
loss_fn = nn.CrossEntropyLoss()
```

숫자가 정확하게 nn.LogSoftmax 및 nn.NLLLoss와 동일함을 확인하자. 한 번에 모든 걸 하면 편하겠지만, 모델의 출력을 확률(혹은 로그 확률)로 해석 가능하지 못하는 단점이 하나 있다. 그래서 이를 구하기 위해 출력을 다시 소프트맥스에 명시적으로 연결할 필요가 있다.

모델을 훈련시키고 검증셋에 대해 정확도(0.802000)를 평가하므로 모델이 클수록 정확도가 올라가야 하지만 생각보다 많이 향상되지는 않음을 발견했다. 훈련셋에 대해서는 거의 완벽한 정확도(0.998100)를 보여줬다. 이는 무슨 의미일까? 두 경우 모두 모델이 과적합하는 상황이다. 완전 연결 모델은 훈련셋을 암기하는 방식으로 비행기와 새를 구별하고 있어서 더 큰 모델을 선택하더라도 검증셋의 성능은 그다지 좋지 못하다.

파이토치는 `nn.Model`의 `parameters()` 메소드(옵티마이저에 파라미터를 제공하기 위해 우리가 이전에 사용했던 것과 동일한 메소드)를 통해 모델이 얼마나 많은 파라미터를 가지고 있는지를 빠르게 확인할 수 있다. 각 텐서 객체에 얼마나 많은 요소가 있는지 확인하려면 `numel` 메소드를 사용한다. 그리고 얻은 값을 합산하면 전체 수를 파악할 수 있다. 파라미터 수를 세려면 파라미터에 대해 `requires_grad`가 `True`로 설정되었는지도 확인해야 한다. 전체 모델 크기 대비 학습 가능한 파라미터의 수는 구별되어야 하기 때문이다. 다음 코드를 보자.


```
# In[7]:
numel_list = [p.numel()
               for p in connected_model.parameters()
               if p.requires_grad == True]
sum(numel_list), numel_list

# Out[7]:
(3737474, [3145728, 1024, 524288, 512, 65536, 128, 256, 2])
```

세상에나, 370만 개가 넘는 파라미터라니! 조그마한 입력 이미지를 다룰 만한 작은 신경망이라고 보긴 어렵지 않은가? 심지어 첫 번째 신경망도 꽤 크다.

```
# In[9]:
numel_list = [p.numel() for p in first_model.parameters()]
sum(numel_list), numel_list

# Out[9]:
(1574402, [1572864, 512, 1024, 2])
```

첫 번째 모델의 파라미터 수는 두 번째 모델의 절반 가까이 된다. 각 파라미터 크기를 보면 어디가 문제인지 알 수 있을 것 같다. 첫 번째 모듈에는 150만 개의 파라미터가 있다. 전체 신경망에서 1,024개의 출력 피처가 있는데, 이로 인해 첫 번째 선형 모듈은 300만 개의 파라미터를 가진다. 이런 부분은 예상할 수 있어야 한다. 선형 계층이 $y = \text{weight} * x + \text{bias}$ 를 가지며 만일 x 의 길이가 3,072(일단 배치 차원은 무시하자)이고 y 가 1,024라면 weight 텐서는 $1,024 \times 3,072$ 가 되고 bias 크기도 1,024가 되어야 한다. 그래서 $1,024 \times 3,072 + 1,024 = 3,146,752$ 로 앞에서 얻은 값과 일치한다. 이 수치는 다음과 같은 코드로 확인할 수도 있다.

```
# In[10]:  
linear = nn.Linear(3072, 1024)  
  
linear.weight.shape, linear.bias.shape  
  
# Out[10]:  
(torch.Size([1024, 3072]), torch.Size([1024]))
```

여기서 얻은 결과는 어떤 의미일까? 우리의 신경망이 픽셀 수에 따라 확장 가능하기 어렵다는 뜻이다. 이미지가 $1,024 \times 1,024$ RGB 이미지라면 어떻게 될까? 입력값이 이미 310만 개다. (현재의 분류기로는 동작하지 않겠지만) 1,024개의 은닉된 피처로 들어갈 것이고 대략 30억 개의 파라미터를 가지게 된다. 32비트 부동소수점을 사용하면 기울기 계산을 위해 더 많은 연산과 저장 공간이 필요한 두 번째 계층을 거치기도 전에 12기가 램 영역이 필요하게 된다. 오늘날 최신 GPU 리소스를 이미 다 채워버린다.

7.2.7 완전 연결의 한계

그림 7.15를 보며 이미지를 1차원으로 놓고 선형 모듈을 사용하는 방식에 대해 좀 더 곰곰이 생각해보자. 입력으로 들어오는 RGB 이미지의 모든 정보 하나하나를 사용하며 출력 피쳐 하나하나에 대해 각 정보의 선형 조합^{combination}을 계산한다. 그리고 현재의 식별 작업에 잠재적으로 필요하다는 가정하에 픽셀 하나마다 다른 모든 픽셀과의 조합을 고려하고 있지만 상대적으로 가까운 위치나 먼 위치에 있는 점을 사용하지 않고 이미지를 하나의 큰 숫자 벡터로 취급하고 있을 뿐이다.

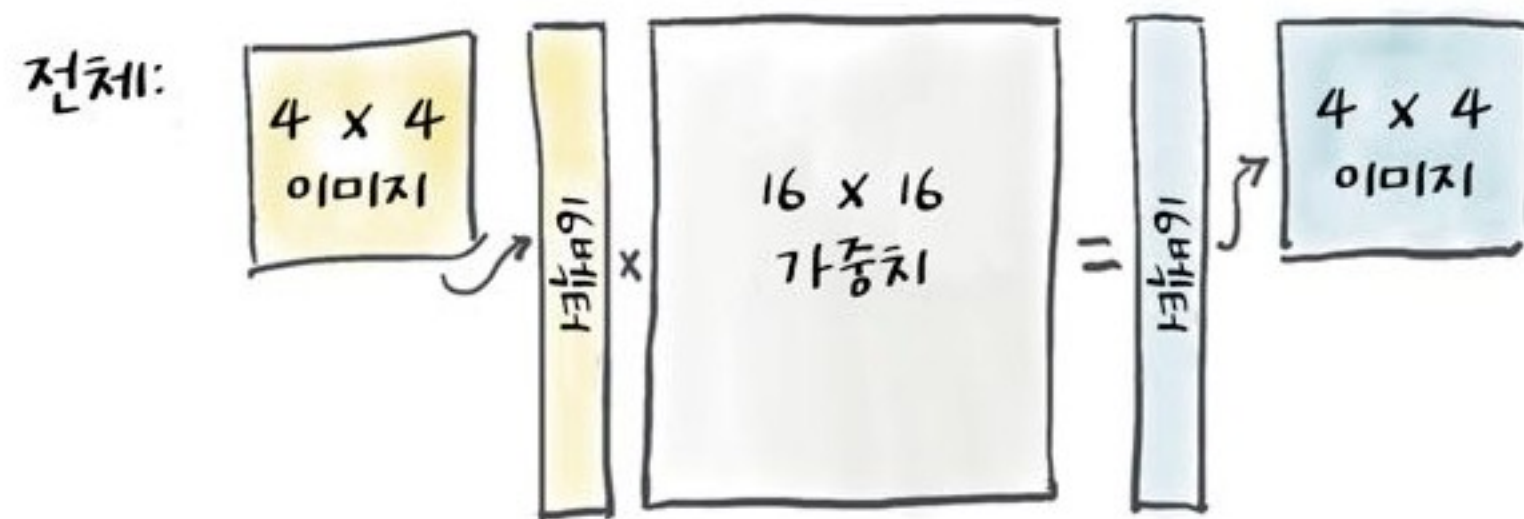
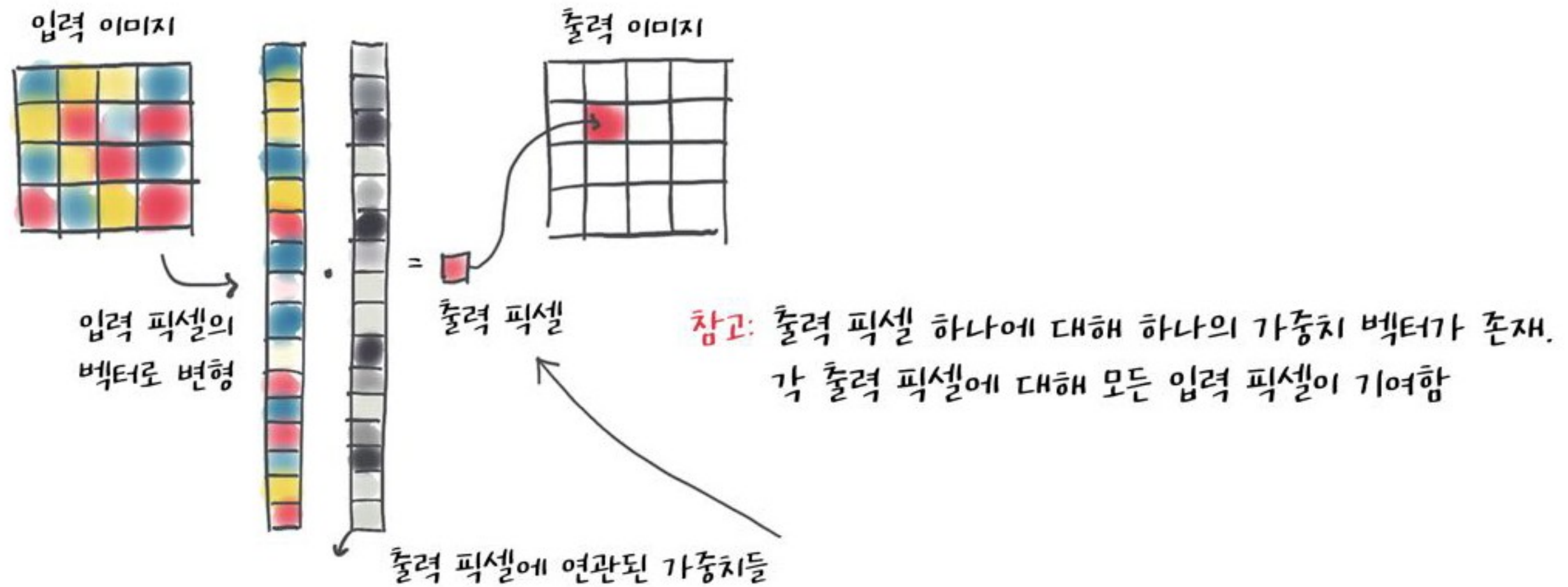


그림 7.15 입력 이미지에 대해 완전 연결 모듈을 사용하는 경우. 출력 요소 하나를 만들기 위해 모든 입력 픽셀이 조합되는 형태다.

32×32 이미지에 잡힌 하늘을 나는 비행기는 파란색 배경에 어두운 십자가 모양과 거의 유사할 것이다. 그림 7.15의 완전 연결 신경망은 학습을 통해 픽셀 0,1은 어둡고 픽셀 1,1도 어둡고, 하는 식으로 인식하면 비행기일 가능성이 높다고 판단할 것이다. 그림 7.16의 상단에 이를 나타내 봤다. 하지만 비행기를 한 픽셀 이상 위치를 이동하면 하단에 그린 것처럼 각 픽셀 간의 관계를 처음부터 다시 학습해야 한다. 이 경우 비행기는 픽셀 0,2가 어둡고 픽셀 1,2도 어둡고, 하는 식으로 학습한다. 전문 용어로 설명하자면 완전 연결 신경망은 **평행이동 불변성**translation invariance이 없다. 즉 위치 4,4에서 시작하는 스피트파이어¹⁰를 인식하도록 훈련된 신경망은 위치 8,8에 나타난 동일한 스피트파이어를 인식할 수 없다는 의미다. 이를 보완하려면 데이터셋을 **증강**해서, 즉 훈련 때 이미지를 랜덤하게 평행이동시켜서 이미지의 모든 영역에서 스피트파이어를 보게 해야 하고 이 작업을 데이터셋의 모든 이미지에 대해 수행해야 한다(torchvision.transforms로 평행이동된 이미지를 원래 데이터에 붙여 넣을 수 있다). 하지만 이러한 데이터 증강data augmentation 전략은 상당한 비용이 든다. 파라미터에 해당하는 은닉된 피처의 수가 이렇게 평행이동시킨 복제본에 대한 모든 정보를 보관하기 충분할 만큼 커야 하기 때문이다.

비행기

0	1	0	0
1	1	1	0
0	1	0	0
0	0	0	0



0
1
0
1
1
1
0
0
0
1
0
0
0
0
0
0
0
0

x

0
1
0
1
1
1
0
0
0
1
0
0
0
0
0
0
0

+

출력

	5		

비행기 (평행이동)

0	0	1	0
0	1	1	1
0	0	1	0
0	0	0	0



0
0
1
0
0
1
0
1
0
0
0
1
0
0
0
0
0

x

0
1
0
1
1
1
0
0
0
1
0
0
0
0
0
0
0

+

가중치

		1	

이번 7장에서 우리는 데이터셋과 모델, 훈련 루프를 가지고 모델을 학습했다. 하지만 주어진 문제와 사용한 신경망 구조가 맞지 않아 모델이 일반적인 특징을 잡아내어 학습하지 못하고 오히려 훈련 데이터에 과적합됐다.

그래서 이미지 내 각 픽셀이 다른 모든 픽셀과 연관되는 모델을 만들었고, 공간상의 배치는 고려하지 않았다. 서로 가까이 있는 픽셀은 관련이 있다는 합리적인 가정을 해보았다. 따라서 우리가 훈련시킨 분류기는 평행이동에 대응할 수 없었다. 이것으로 검증셋에서 잘 동작하려면 원본 이미지를 평행이동시켜 만든 많은 복제본에 대해 동작하는 더 큰 용량의 모델이 필요하게 됐다. 더 좋은 방법은 없을까?

물론, 어떤 책에서 이런 질문을 던질 때엔 다소 과장이 섞이기 마련이다. 현재 직면한 문제를 해결하려면 모델이 컨볼루션층을 사용하도록 변경해야 하며 이에 관해서는 다음 8장에서 다룰 예정이다.

7.3 결론

이번 7장에서 우리는 간단한 분류 문제를 풀어봤다. 데이터셋에서 모델을 구성하는 동안 훈련 루프에서 적당한 손실값을 정의하고 최소화해봤다. 이들은 앞으로 여러분이 파이토치를 사용하며 다루게 될 표준적인 도구들이다. 또한 이 장에서 배운 기술은 앞으로 파이토치와 함께 하는 동안 매우 유용할 것이라고 생각한다.

또한 우리는 모델을 만들었고, 여기서 중대한 결점도 찾아봤다. 2차원 이미지를 1차원 데이터처럼 취급했는데 문제시되는 평행이동 불변성에 대응할 만한 자연스러운 방법은 찾아내지 못했다. 다음 8장에서 여러분은 훨씬 나은 결과를 얻기 위해 이미지 데이터의 2차원 특성을 어떻게 활용하면 될지를 배울 것이다.¹¹

지금까지 배운 것은 평행이동 불변성을 포함하는 경우가 아니라면 바로 데이터를 적용해서 활용할 수 있다. 예를 들어 7장에서 배운 내용을 4장에서 살펴본 테이블이나 시계열 데이터에 활용하면 훌륭하게 동작한다. 적당하게 표현된 상태라면 텍스트 데이터에 대해서도 어느 정도는 사용 가능할 것이다.¹²

7.5 핵심 요약

- 컴퓨터 비전은 딥러닝의 가장 주요한 응용 분야 중 하나다.
- 레이블된 여러 데이터셋이 공개되어 있으며 torchvision을 통해 접근 가능하다.
- Dataset과 DataLoader는 단순하면서도 효과적인 데이터 로딩과 샘플링을 위한 추상화를 제공한다.
- 분류 작업에서 소프트맥스 함수를 신경망의 출력단에 사용하면 결과값을 확률로 해석할 때 필요한 요구사항을 만족할 수 있다. 이 경우 분류를 위한 이상적인 손실 함수는 소프트맥스의 출력을 ‘음이 아닌 로그 가능도 함수non-negative log likelihood function’의 입력으로 사용해 얻을 수 있다. 파이토치에서 소프트맥스와 이런 손실값의 조합을 크로스엔트로피라 부른다.
- 이미지를 픽셀값의 벡터로 간주하고 일반 숫자 데이터처럼 완전 연결 신경망으로 다뤄도 뭐라 할 사람은 없다. 하지만 이러면 데이터 내에서 공간적인 관계를 활용하기가 어려워진다.
- 단순한 모델은 nn.Sequential로 만들 수 있다.